

Міністерство освіти і науки України
Національний Університет “Львівська Політехніка”



Лабораторна робота
з дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав:
студент гр. ІХ-22
Ференц Роман

Прийняв:
Гордійчук-Бублівська О.В

Лабораторна робота 2. Моделювання роботи телекомунікаційної мережі за допомогою ООП

Тема: Моделювання телекомунікаційної мережі з використанням генераторів подій, протоколів TCP/UDP, асинхронного моделювання. Розробка графічної візуалізації мережі та аналіз продуктивності..

Мета: Ознайомлення з основами моделювання телекомунікаційних мереж. Вивчення використання об'єктно-орієнтованого програмування для моделювання мережі. Моделювання різних топологій мереж (зірка, кільцева, деревоподібна, сіткова (mesh), комбінована тощо). Симуляція передачі пакетів між вузлами та маршрутизаторами з використанням TCP та UDP. Вимірювання продуктивності мережі: швидкість передачі, статистика втрат, затримки передачі. Побудова візуалізації для графічного відображення структури та роботи мережі.

Завдання:

```
▶ import asyncio
import random
import time
import networkx as nx
import matplotlib.pyplot as plt

class Node:
    def __init__(self, name):
        self.name = name
        self.neighbors = []

    def connect(self, other):
        if other not in self.neighbors:
            self.neighbors.append(other)
            other.neighbors.append(self)

    async def send_packet(self, packet, net):
        await asyncio.sleep(random.uniform(0.05, 0.2))

        if random.random() < net.loss:
            net.lost += 1
            return

        await self.forward_packet(packet, net)

    async def forward_packet(self, packet, net):
        if self == packet.destination:
            return
```



c.

```
        if self in packet.path:
            return

        packet.path.append(self)

        for n in self.neighbors:
            if n not in packet.path:
                await n.send_packet(packet, net)

class Router(Node):
    def __init__(self, name):
        super().__init__(name)

class Packet:
    def __init__(self, source, destination, protocol):
        self.source = source
        self.destination = destination
        self.protocol = protocol
        self.size = random.randint(100, 500)
        self.path = []

class TCP:
    name = "TCP"

    @staticmethod
    async def send(source, destination, net):
        packet = Packet(source, destination, "TCP")
        await source.send_packet(packet, net)
```



```
class UDP:
    name = "UDP"

    @staticmethod
    async def send(source, destination, net):
        packet = Packet(source, destination, "UDP")
        await source.send_packet(packet, net)

class Network:
    def __init__(self, topology):
        self.nodes = []
        self.topology = topology
        self.loss = random.uniform(0.10, 0.15)
        self.sent = 0
        self.lost = 0
        self.total_time = 0

    def create_star(self):
        center = Router("Router")
        pcs = [Node(f"PC{i}") for i in range(1, 6)]
        self.nodes = [center] + pcs

        for pc in pcs:
            center.connect(pc)

    def create_mesh(self):
        pcs = [Node(f"PC{i}") for i in range(1, 6)]
        self.nodes = pcs

        for i in range(len(pcs)):
            for j in range(i + 1, len(pcs)):
                pcs[i].connect(pcs[j])
```



```
async def simulate(self, protocol, count=10):
    for _ in range(count):
        s, d = random.sample(self.nodes, 2)
        start = time.time()
        self.sent += 1
        await protocol.send(s, d, self)
        self.total_time += time.time() - start

def stats(self):
    avg = self.total_time / self.sent
    loss_percent = (self.lost / self.sent) * 100
    speed = (self.sent - self.lost) / self.total_time

    print("\n---", self.topology, "---")
    print("Avg time:", round(avg, 4))
    print("Loss %:", round(loss_percent, 2))
    print("Throughput:", round(speed, 2))

def draw(self):
    G = nx.Graph()
    for node in self.nodes:
        for n in node.neighbors:
            G.add_edge(node.name, n.name)

    pos = nx.spring_layout(G)
    nx.draw(G, pos, with_labels=True, node_size=2000)
    plt.title(self.topology)
    plt.show()
```

```
async def main():

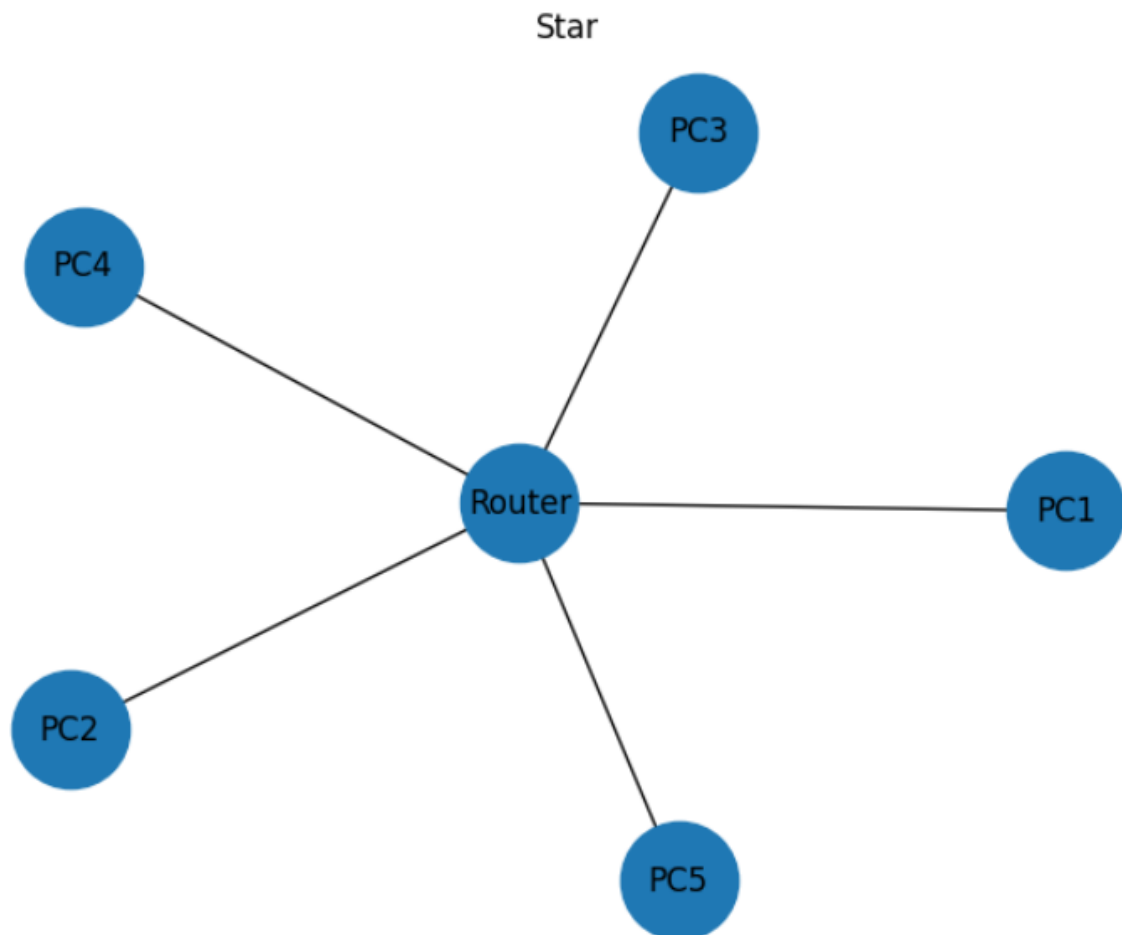
    net1 = Network("Star")
    net1.create_star()
    await net1.simulate(TCP)
    net1.stats()
    net1.draw()

    net2 = Network("Mesh")
    net2.create_mesh()
    await net2.simulate(UDP)
    net2.stats()
    net2.draw()

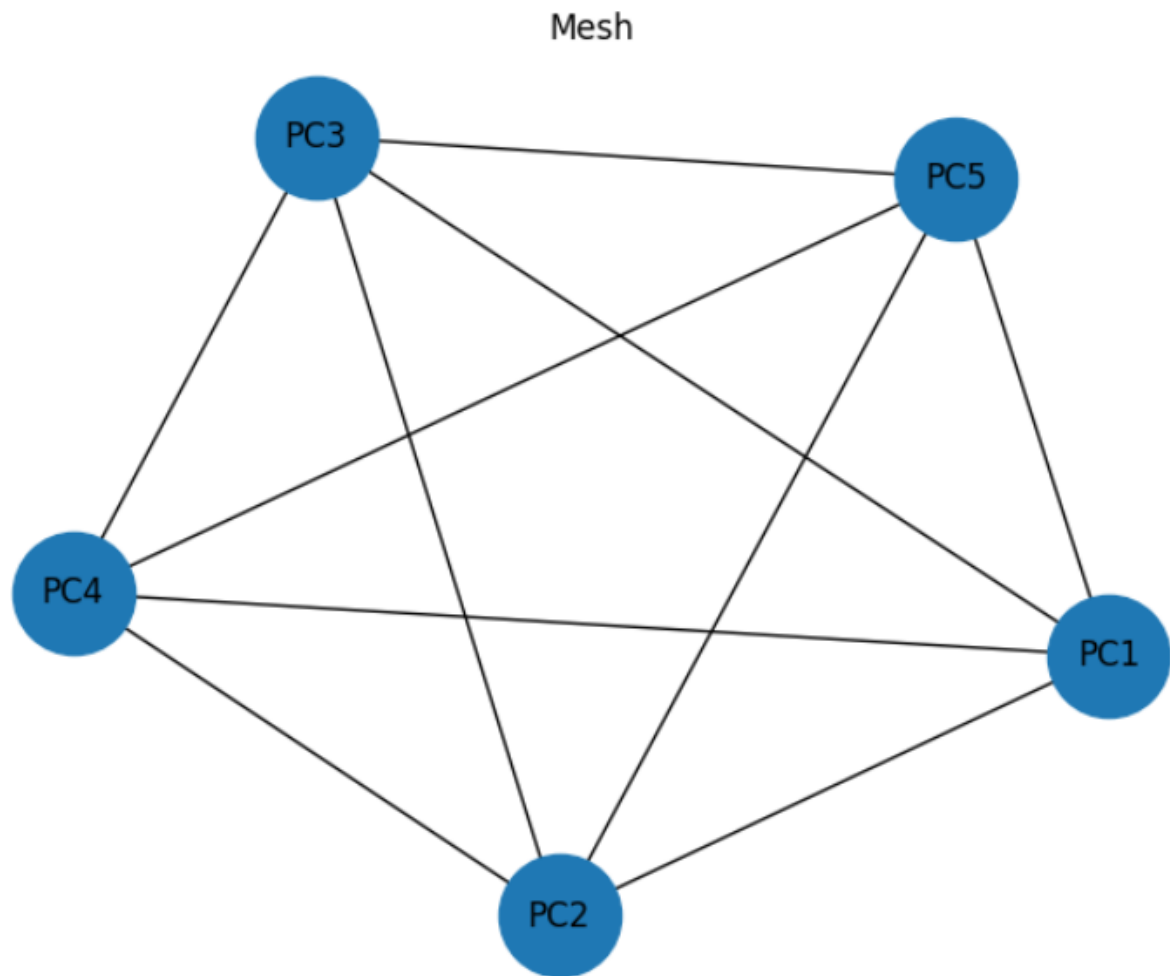
await main()
```

Результат:

```
, --- Star ---
Avg time: 0.6325
Loss %: 90.0
Throughput: 0.16
```



--- Mesh ---
Avg time: 1.0647
Loss %: 60.0
Throughput: 0.38



Висновок:

У ході лабораторної роботи було змодельовано телекомунікаційну мережу з використанням об'єктно-орієнтованого підходу та асинхронної передачі пакетів. Проведене порівняння зіркової та сіткової топологій показало, що тип структури мережі впливає на затримку, втрати пакетів і пропускну здатність. Отримані результати підтвердили, що сіткова топологія забезпечує кращу надійність і швидкість передачі даних у порівнянні із зірковою.